

Digital Security Lock

A Sequential Logic Design Project

Design — 12-bit serial sequence detection lock

Derive — FSM, counters, and output logic from first principles

Implement — Full VHDL design synthesized to Basys3 FPGA

Demonstrate — Hardware operation and analytical understanding

One Submission

Full Derivation Required

Vivado + Basys3

VHDL

A Complete Sequential Logic Design — From Derivation to Hardware

This project asks you to design, analyze, simulate, and implement a **digital security lock** — a system that accepts a serial binary stream and decides whether to open based on the pattern content of that stream. The design integrates a finite-state machine, modular counters, and combinational output logic into a single working mechanism, culminating in a hardware demonstration on the Basys3 FPGA board.

You will be expected to understand, derive, and document every aspect of your design. A working implementation without analytical derivation work is not a complete submission. A correct derivation that never reaches hardware is not complete either. Both are required and graded separately.

This is a single project with one final submission. There are no milestone check-ins. Plan your work accordingly.

What you will submit

One technical report (PDF) · VHDL source files · Simulation waveform evidence · Hardware demonstration (live or recorded). All four components are required. Details in Section 5.

1 Background and Context

This project draws on three families of digital logic concepts covered this semester: **Moore finite-state machine design**, **synchronous counter construction**, and **combinational output logic**. Each is individually well-defined. The challenge here is using them together — understanding how the output of one subsystem becomes the enable signal of another, how reset propagates through a composed system, and how to verify that the integrated design behaves correctly before committing to hardware.

Before beginning, review the following topics in your textbook: Moore and Mealy FSM design procedure, state minimization and equivalent-state analysis, D flip-flop input equations, synchronous counter design with custom moduli, Boolean algebra and Karnaugh maps, and synchronous reset behavior. The project assumes proficiency with all of these.

2 System Specification

2.1 Behavioral Description

The security lock accepts a **serial binary input stream**. When the system is enabled, each rising clock edge causes one bit to be sampled and processed. A complete attempt consists of exactly **12 consecutive bits**.

Those 12 bits are interpreted as **four consecutive, non-overlapping 3-bit groups**. The first group is bits 1–3, the second is bits 4–6, the third is bits 7–9, and the fourth is bits 10–12. Group boundaries are fixed by position in the stream, not by content.

A group is **valid** if and only if its three bits — in the exact order received — form one of: **010** or **100**. Any other 3-bit combination is invalid.

The lock **opens** — asserting OPEN — if and only if **all four groups** are valid. If any group is invalid when the 12th bit is received, the lock stays closed, the system resets, and a new 12-bit window begins. Once OPEN is asserted it remains asserted until an explicit reset is applied.

■ 2.2 System Interface

Signal	Dir	Width	Description
X	In	1 bit	Serial data input. One bit sampled per rising clock edge when EN = 1.
CLK	In	1 bit	System clock. All sequential elements triggered on the rising edge.
RST	In	1 bit	Synchronous reset. All state elements return to their initial state on the next rising edge.
EN	In	1 bit	System enable. When deasserted, no state changes occur. System holds current state.
OPEN	Out	1 bit	Lock open indicator. Asserted after a complete valid 12-bit window. Remains asserted until reset.
WINDOW_DONE	Out	1 bit	Window complete indicator. Asserted when 12 bits have been received. May drive an LED.
COUNT_OUT	Out	4 bits	Current valid detection count. Drives the 7-segment display during operation.

Table 1. System interface signals. Internal signal names within your design are your own choice.

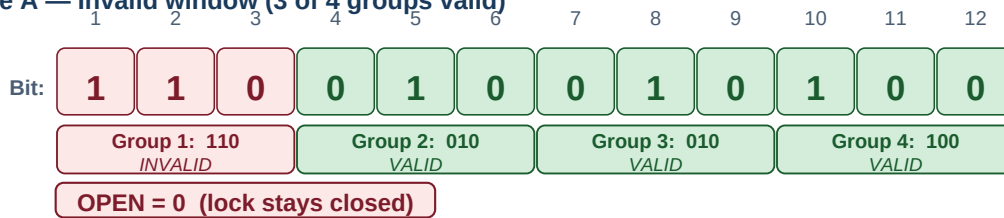
On naming

The signal names in Table 1 describe function, not required HDL port names. Choose clear, consistent names in your VHDL. Whatever names you use, document them in your report and ensure your top-level entity port list matches your constraints file exactly.

■ 2.3 Annotated Examples

The two examples below illustrate the system's behavior on complete 12-bit streams. They define the behavioral specification, not the implementation.

Example A — Invalid window (3 of 4 groups valid)



Example B — Valid window (all 4 groups valid)

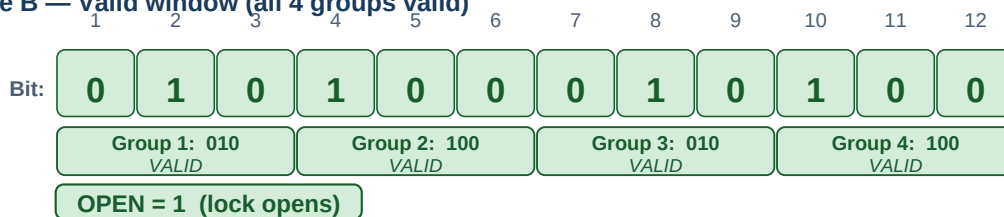


Figure 1. Annotated 12-bit stream examples. Group boundaries are fixed by position. OPEN asserts only when all four groups are valid.

2.4 Behavioral Constraints

The following constraints must be satisfied by your design:

- The system processes exactly **one bit per rising clock edge** when EN = 1.
- Group boundaries are determined solely by **bit position**. The stream does not re-align based on detected patterns.
- Detection is **non-overlapping**. Once a 3-bit group is evaluated, the next bit begins a new group. No bit is shared between adjacent groups.
- All state-holding elements reset **synchronously** on RST = 1. Reset takes effect on the next rising clock edge.
- OPEN remains asserted **until reset** is applied.
- When EN = 0, no sequential state changes. The system holds its current state.
- The valid detection count on the display must be consistent with the number of valid groups confirmed in the current window.

3 Design Requirements

This section describes what you must design and what understanding you must demonstrate. It is organized around the natural subsystems of the lock. Each subsystem has a defined behavioral role; *how* you implement that role is your design decision. The section will tell you what artifacts to derive and what reasoning to document — it will not show you the solution.

3.1 Sequence Detector

The core of the lock is a finite-state machine that watches the incoming bit stream and recognizes valid 3-bit groups. Your FSM must be designed as a **Moore machine**: the output depends only on the current state, not the current input. The FSM operates on one serial bit per clock cycle and must correctly identify both **010** and **100** as valid patterns.

Non-overlapping detection: after the FSM completes evaluation of a 3-bit group — valid or not — it returns to a defined starting condition and begins evaluating the next group from scratch. No partial sequence state carries over from one group to the next.

Where to start

What does the FSM need to remember between clock cycles in order to recognize these two patterns? The answer to that question determines how many states you need. Think about what distinguishes one partial input sequence from another before you draw your first state.

What you must derive and include in your report:

- A **state diagram** — all states, all transitions for $X = 0$ and $X = 1$ from every state, and the output value at each state.
- A fully populated **state table** with columns for current state, input, next state, and output.
- A **state minimization analysis**. Determine whether any states are equivalent and can be merged. If no reduction is possible, document that conclusion and explain why.
- A **state assignment** — a binary encoding for each state, with a brief justification.
- **D flip-flop input equations** for each state bit, derived from your state table. Show your derivation work (K-map or algebraic).
- The **output equation** for the detection signal Z . $Z = 1$ indicates a valid group has been completed.

Your FSM must handle every combination of current state and input bit. States that appear unreachable must still be accounted for.

■ 3.2 Detection Counter

Each time the FSM asserts a valid detection ($Z = 1$), the system must count that event. The detection counter tracks how many valid 3-bit groups have been confirmed within the current window.

What you must design and explain:

- The **counting sequence** — what range does this counter need, and why?
- The **combinational logic** that controls when the counter increments.
- The **ceiling detection logic** — what signal indicates the upper limit has been reached, and how is that signal used?
- How this counter responds to **system reset** and to the window-done condition from Section 3.3.

■ 3.3 Bit Counter and Window Control

The system must also track how many bits have been received in the current 12-bit window. This counter counts received bits and generates a window-complete signal when 12 bits have been processed.

What you must design and explain:

- The **counting sequence** — what modulus does this counter require?
- The **enable logic** — what causes this counter to increment?
- The **window-complete condition** — when exactly does it trigger, and what does it cause elsewhere in the system?
- How **EN = 0** and **RST = 1** affect this counter.

Timing question

When the 12th bit arrives, your system must decide whether to open the lock. Consider what state the detection counter is in at that exact moment, and whether the decision happens on the same clock edge as the 12th bit or on the one after. Your report must clearly answer this.

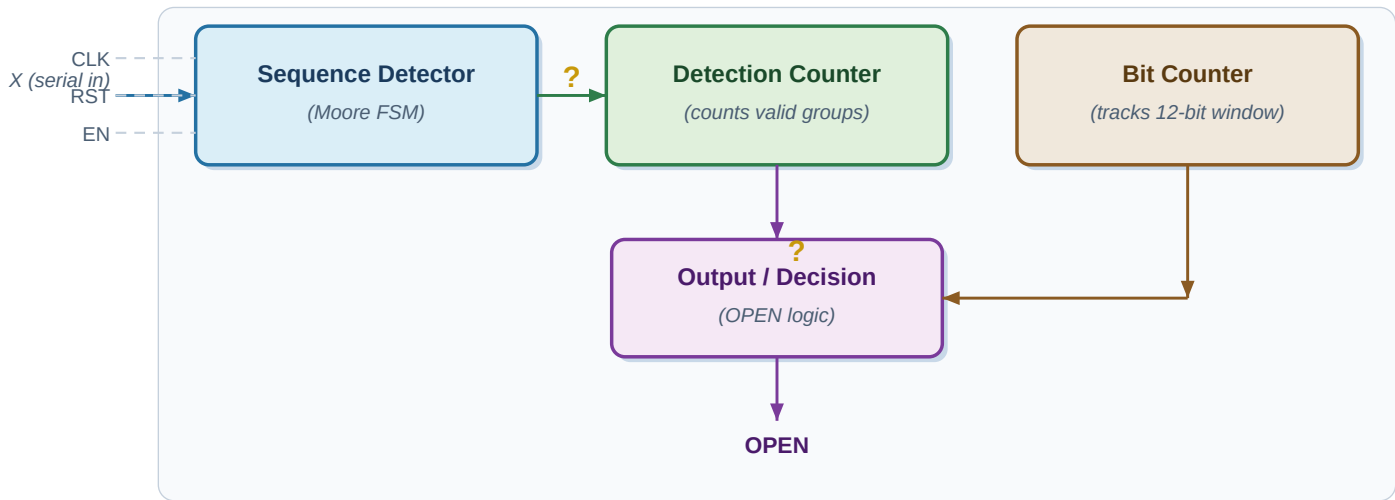
■ 3.4 System Integration and Output Logic

The sequence detector and the two counters do not operate in isolation. You must design the **combinational logic** that combines their outputs to produce the final OPEN signal, and the signal connections that allow subsystems to interact correctly.

Questions your design must answer and document:

- Under what **exact logical conditions** does OPEN assert? Express this as a Boolean function.
- What signals are routed from the FSM to the detection counter?
- What signals are routed from the bit counter to the rest of the system?

- How does the window-complete signal interact with the detection counter reset for the next window?



Partial diagram — signal names and connections intentionally omitted. Your submission must include a fully annotated version.

Figure 2. Partial conceptual block diagram. Signal names, port details, and internal connections are intentionally omitted. Your submission must include a fully annotated version with all inter-subsystem signals named and described.

Your report must include a **completed top-level block diagram** that names every inter-subsystem signal, shows direction, and is consistent with your VHDL implementation. Figure 2 is a prompt, not the answer.

■ 3.5 Reset and Enable Behavior

Reset and enable logic must be explicitly addressed in your design. Vague descriptions are not acceptable.

- **Reset (RST = 1)**: Every state-holding element returns to a precisely defined initial state. Specify the reset state of each subsystem. Reset is synchronous throughout.
- **Enable (EN = 0)**: No sequential state may change. Define precisely what this means for each subsystem.
- Consider what happens if RST and EN are both asserted simultaneously. Your design must have a defined, deterministic behavior for this case.

4 Implementation Requirements

4.1 VHDL Implementation

Your complete design must be implemented in **VHDL** and must be synthesizable. Behavioral VHDL is acceptable, but your report must demonstrate that the behavioral description is logically consistent with your manually derived state assignments and equations.

Your design must, at minimum, separate the sequence detector FSM into its own VHDL entity or component. Additional modularization — for example, giving the counter logic its own entity — is strongly encouraged and improves readability and testability. The subsystem structure you document in your report should be visible in your implementation.

- Use synchronous reset throughout: if rising_edge(CLK) then if RST = '1' then ...
- Port names in your top-level entity must match your constraints file exactly.
- Each file should contain one entity/architecture pair with clear, consistent naming.

Behavioral vs. structural VHDL

A process with a case statement encoding next-state logic is perfectly acceptable. What matters is that you can show the correspondence between that code and the state table and equations you derived. If your code says one thing and your report says another, that is a problem regardless of whether the board works.

4.2 Vivado Workflow

Create a Vivado project targeting the **Basys3 Artix-7 FPGA** (part: xc7a35tcpg236-1). Add all VHDL source files and a constraints file (.xdc) with your pin assignments. Run synthesis and implementation. Generate a bitstream and program the board.

Before programming the board, run a **behavioral simulation** in Vivado's integrated simulator. Your simulation must include at minimum: the FSM subsystem in isolation and the full integrated system on both a valid and an invalid 12-bit sequence. Save annotated waveform screenshots for your report.

4.3 Hardware Demonstration

Your implementation must be demonstrated on the Basys3 board and must clearly show all of the following:

- A **valid 12-bit sequence** entered in full, causing OPEN to assert.
- An **invalid 12-bit sequence** entered in full, leaving OPEN deasserted.
- The **detection count advancing correctly** on the 7-segment display as valid groups are recognized.

- **Correct reset behavior:** RST clears OPEN and returns the system to its initial state.

Demonstration may be a **live in-class demo** or a **clearly recorded video**. Recordings must be unambiguous — show the board in frame, show each bit being entered, and show the output state at the end of the sequence.

■ 4.4 7-Segment Display

The detection count (COUNT_OUT) must be displayed on at least one digit of the Basys3's seven-segment display during operation. A BCD-to-segment decoder (SSD driver) may be used as a provided supporting component — you are not required to derive it from scratch. It must, however, be correctly wired in your VHDL and produce correct output on the board.

■ 4.5 RedByte (Optional Design Aid)

RedByte provides a visual logic design environment that may be useful for organizing subsystem structure, verifying interface connections, and visualizing state behavior before writing VHDL. The *Final Project: Security Lock Starter* example in RedByte's library includes the correct Basys3 pin mappings and three pre-loaded verification sequences — one invalid stream and two valid streams — that you can use to check design logic interactively.

RedByte is not required and is not graded

RedByte usage is entirely optional. Your grade depends on your analytical derivation work, your VHDL, your simulation, and your hardware demonstration — not on whether you used RedByte. If your design works correctly on the Basys3 board, that is the implementation standard regardless of what design tools you used.

5 Submission Requirements

One submission package per student. Submit a single compressed archive (.zip) containing all four components below. Submissions missing any component will be graded accordingly.

Component	Format	Required Contents
Technical Report	PDF	Organized per Appendix C. Must include all required design artifacts: state diagram, state table, minimization analysis, equations, counter derivation, integration diagram, simulation results, and hardware test evidence.
VHDL Source Files	.vhd files	All files needed to recreate the Vivado project from scratch. Include the top-level entity, all sub-entities, and the constraints file (.xdc).
Simulation Evidence	Screenshots	Annotated waveform screenshots showing: (1) FSM subsystem correct response to a test sequence, and (2) full system correctly asserting and withholding OPEN.
Hardware Evidence	Video / demo	Unambiguous demonstration of all behaviors in Section 4.3. If recorded, include the video file or a link on the report cover page.

Table 2. Submission components. All four are required.

6 Grading Criteria

Area	Weight	What Is Evaluated
Design Derivation & Analysis	45%	State diagram completeness and correctness. State table. Minimization analysis (result and justification). State assignment. Flip-flop input equations with derivation work shown. Counter logic reasoning. Integration diagram and Open logic derivation. Reset and enable analysis.
VHDL & Simulation	25%	Code is synthesizable. Subsystem structure reflects the design. Simulation demonstrates correct FSM behavior. Full-system simulation correct on valid and invalid sequences. Waveforms are annotated and readable.
Hardware Implementation	20%	All behaviors in Section 4.3 demonstrated on the Basys3. Detection count visible on display. Reset behavior correct. Demonstration is clear and unambiguous.
Report Quality & Completeness	10%	Report follows the structure in Appendix C. Figures and tables are captioned. Explanations are complete and professionally written. VHDL source included as appendix.

Table 3. Grading breakdown.

Grading note

A hardware implementation that works but lacks derivation work will not receive full credit for the 45-point derivation category — those points require documented analytical work regardless of board function. Conversely, a correct derivation that is never implemented earns no hardware points. Both halves are assessed independently.

Appendix A — Basys3 I/O Quick Reference

The table below lists the most commonly used Basys3 I/O resources for this project. Pin numbers are from the Digilent Basys3 Master XDC (v1.2). **Always verify against the official constraints file** — an incorrect pin assignment will prevent synthesis.

Resource	Identifier	Basys3 Pin	Notes
Slide switch 0	SW0	V17	Rightmost switch
Slide switch 1	SW1	V16	
Slide switch 2	SW2	W16	
Slide switch 3	SW3	W17	
Slide switch 4	SW4	W15	Suggested: RST
Slide switch 5	SW5	V15	Suggested: EN
Slide switch 14	SW14	T1	Suggested: X (serial in)
Center pushbutton	BTNC	N17	Suggested: manual CLK
Left pushbutton	BTNL	P17	
Right pushbutton	BTNR	M17	
LED 0	LD0	U16	Suggested: OPEN
LED 1	LD1	E19	Suggested: WINDOW_DONE
7-seg segments CA–CG	CA, CB, CC, CD, CE, CF, CG	W7, W6, U8, V8, U5, V5, U7	Segment cathodes, active low. Provided SSD driver handles encoding.
7-seg decimal point	DP	V7	Active low. Leave unconnected if unused.
7-seg anode 0 (rightmost)	AN0	U2	Active low. Drive low to enable digit.
7-seg anode 1	AN1	U4	Active low
7-seg anode 2	AN2	V4	Active low
7-seg anode 3 (leftmost)	AN3	W4	Active low

On-board 100 MHz clock	CLK100MHZ	W5	Use only if adding a clock divider. Not needed for manual-clock demo.
------------------------	-----------	----	---

Table A-1. Basys3 I/O reference (partial). Verify all pin numbers against the official Digilent Basys3 Master XDC before submitting your constraints file.

Manual clock and debounce

A pushbutton clock is sufficient and strongly recommended for the demo: connect a pushbutton to CLK and press once per bit. Because mechanical buttons bounce, you must condition the signal before using it as a clock — without this, a single press may register as multiple rising edges and corrupt the sequence. A debounce / one-shot module is provided as a supporting component (same status as the SSD driver) — you are not required to design it from scratch, but you must instantiate it correctly between your pushbutton pin and your CLK input.

Appendix B — Design Artifact Format Reference

Use these format conventions for the design artifacts in your report. Consistent, readable formatting is part of the report quality grade.

■ State Diagram

- Each state is a labeled circle. Moore outputs are written inside the circle below the state label (output name = value).
- Transitions are directed arcs labeled with the input value (e.g., $X = 0$ or $X = 1$).
- Mark the initial state with an unlabeled incoming arrow.
- Every state must have a transition defined for every possible input. No dangling states.

■ State Table

Minimum required columns:

- Current State (symbolic name and binary encoding)
- Input X
- Next State (symbolic name and binary encoding)
- Output Z

List every row. Do not omit cases where the input is 0 or 1.

■ Flip-Flop Input Equations

- Show the K-map or algebraic derivation for each D input equation.
- Label minterms clearly. Circle groups on K-maps.
- Write final equations in sum-of-products or product-of-sums form.

■ Counter Logic

- Describe the counting sequence as a truth table or state sequence.
- Show the combinational logic (Boolean expressions or gates) that produces the enable and load/reset control signals.

Appendix C — Report Format Guide

Your technical report must follow the section order below. Use clear section headings matching this list. All figures and tables must be captioned and referenced in the text.

Sec.	Title	Required Contents
Cover	Title Page	Project title, course number, date, student name, link or note for video demo if applicable.
1	Introduction	Brief description of the system and what the report covers. 1–2 paragraphs.
2	System Specification	Completed top-level block diagram (fully annotated). Final port list with signal names matching your VHDL.
3	Sequence Detector Design	State diagram. State table. Minimization analysis. State assignment. D flip-flop input equations with derivation work shown. Output equation Z.
4	Counter Design	Detection counter: counting sequence, enable logic, ceiling detection. Bit counter: counting sequence, window-complete logic, system interaction.
5	System Integration	Open condition derivation. Enable and reset analysis. Explanation of inter-subsystem signal connections.
6	VHDL Implementation	Entity/architecture overview. Key design decisions. Mapping between VHDL and derived equations.
7	Simulation Results	Annotated waveforms. FSM subsystem test. Full-system test on valid and invalid sequences.
8	Hardware Test and Results	Test sequences used. Step-by-step account of what was entered and what was observed. Results match simulation.
9	Conclusion	Brief reflection on what was learned and any challenges. 1–2 paragraphs.
App.	VHDL Source Code	Complete VHDL listing for all entities in monospace font. Include constraints file (.xdc).

Table C-1. Required report section order and contents.